LAPORAN

WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)

SEMESTER III



BKPM Acara 5 Routing, .Htaccess, Request Dan Response

NAMA:

AZIRA MUTIA GANI

NIM:

E41250290

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Dalam pembuatan aplikasi web, diperlukan suatu mekanisme yang dapat mengatur permintaan pengguna berdasarkan URL yang diakses. Pada aplikasi PHP sederhana, pengguna dapat mengakses file secara langsung melalui URL sehingga alamat yang ditampilkan menjadi kurang rapi dan dapat memperlihatkan struktur file aplikasi.

Untuk mengatasi hal tersebut, dapat digunakan routing dan `.htaccess`. Routing digunakan untuk menentukan controller dan method yang dijalankan berdasarkan URL yang diminta oleh pengguna. Sementara itu, `.htaccess` digunakan untuk mengarahkan permintaan URL ke satu file utama atau *front controller* sehingga URL aplikasi menjadi lebih bersih.

Pada praktikum ini dilakukan implementasi routing sederhana menggunakan PHP, `.htaccess`, serta konsep front controller. Implementasi dilakukan pada aplikasi Si-Akademik dengan beberapa URL seperti `/`, `/mahasiswa`, dan `/mahasiswa/create`. Selain itu, dilakukan juga tugas mandiri berupa penambahan parameter URL pada `/mahasiswa/5`.

1.2 Tujuan

Praktikum ini bertujuan untuk memahami konsep routing pada aplikasi web berbasis PHP serta memahami fungsi `.htaccess` dalam mengatur URL. Praktikum juga bertujuan agar mahasiswa mampu membuat *front controller* menggunakan `index.php`, menghubungkan URL dengan controller dan method, serta menerapkan parameter sederhana pada URL.

BAB 2. LANDASAN TEORI

2.1 Routing

Routing merupakan mekanisme yang digunakan untuk menentukan proses atau halaman yang akan dijalankan berdasarkan URL yang diakses oleh pengguna. Routing membantu mengarahkan permintaan dari browser ke bagian aplikasi yang sesuai sehingga pengelolaan halaman menjadi lebih terstruktur.

Dalam aplikasi Sistem Informasi Akademik, routing dapat digunakan untuk mengatur akses ke halaman beranda, login, daftar mahasiswa, dan detail mahasiswa. Dengan adanya routing, setiap URL dapat diarahkan ke proses yang sesuai dengan kebutuhan aplikasi.

2.2 Controller

Controller merupakan bagian aplikasi yang bertugas menerima dan mengelola permintaan pengguna. Controller mengatur alur pemrosesan aplikasi, memanggil Model apabila diperlukan, dan menentukan View yang akan ditampilkan kepada pengguna.

Pada aplikasi Sistem Informasi Akademik, Controller dapat digunakan untuk menangani permintaan terkait data mahasiswa, seperti menampilkan daftar mahasiswa, menambahkan data, mengubah data, dan menampilkan detail mahasiswa. Pemisahan Controller dari View membantu membuat kode program lebih terorganisasi dan mudah dipelihara.

2.3 URL (Uniform Resource Locator)

URL merupakan alamat yang digunakan untuk mengidentifikasi dan mengakses suatu sumber daya di internet maupun server lokal. URL dapat memuat informasi berupa protokol, nama host, jalur halaman, dan parameter tertentu.

Dalam pengembangan aplikasi web, URL digunakan sebagai dasar penentuan halaman atau proses yang akan dijalankan. Pada aplikasi Sistem Informasi Akademik yang dijalankan melalui XAMPP, URL lokal dapat digunakan untuk mengakses halaman aplikasi melalui browser.

2.4 File .htaccess

File `.htaccess` merupakan file konfigurasi yang digunakan pada server Apache untuk mengatur perilaku direktori tertentu. File ini dapat digunakan untuk mengatur penulisan ulang URL (*URL rewriting*), pengalihan halaman, dan aturan akses tertentu sesuai konfigurasi server.

Dalam penerapan routing, `.htaccess` dapat dikonfigurasi agar permintaan URL yang tidak merujuk langsung pada file atau direktori tertentu diteruskan ke satu file utama, seperti `index.php`. Dengan cara tersebut, aplikasi dapat memproses permintaan melalui mekanisme routing yang telah ditentukan.

2.5 Superglobal `$_SERVER` pada PHP

`$_SERVER` merupakan variabel superglobal PHP yang menyediakan informasi mengenai server dan permintaan HTTP yang sedang diproses. Salah satu elemennya adalah `$_SERVER['REQUEST_METHOD']`, yang digunakan untuk mengetahui metode HTTP pada permintaan, misalnya GET atau POST.

Informasi tersebut dapat dimanfaatkan oleh aplikasi untuk membedakan permintaan yang hanya menampilkan halaman dengan permintaan yang mengirimkan data melalui formulir. Penggunaan `$_SERVER` membantu Controller menentukan tindakan yang sesuai terhadap permintaan pengguna.

2.6 Model-View-Controller (MVC)

Model-View-Controller (MVC) merupakan pola arsitektur yang membagi aplikasi menjadi tiga komponen utama, yaitu Model, View, dan Controller. Model bertanggung jawab terhadap pengelolaan data, View berfungsi menampilkan antarmuka kepada pengguna, sedangkan Controller mengatur alur pemrosesan permintaan.

Dalam penerapan routing, Controller menjadi bagian yang menerima permintaan berdasarkan URL dan menentukan proses yang harus dijalankan. Penerapan MVC membantu memisahkan tanggung jawab setiap komponen sehingga aplikasi lebih mudah dikembangkan dan dipelihara.

2.7 XAMPP dan Apache

XAMPP merupakan paket perangkat lunak yang menyediakan lingkungan server lokal untuk pengembangan aplikasi web. Apache berfungsi sebagai web server yang menerima permintaan browser dan memproses permintaan tersebut melalui konfigurasi serta program yang sesuai.

Dalam praktikum Acara 5, XAMPP digunakan untuk menjalankan aplikasi PHP secara lokal. Apache berperan dalam menangani permintaan URL dan menerapkan konfigurasi `.htaccess`, sehingga routing aplikasi dapat diuji melalui browser sebelum diterapkan pada server online.

BAB 3. PELAKSANAAN PRAKTIKUM

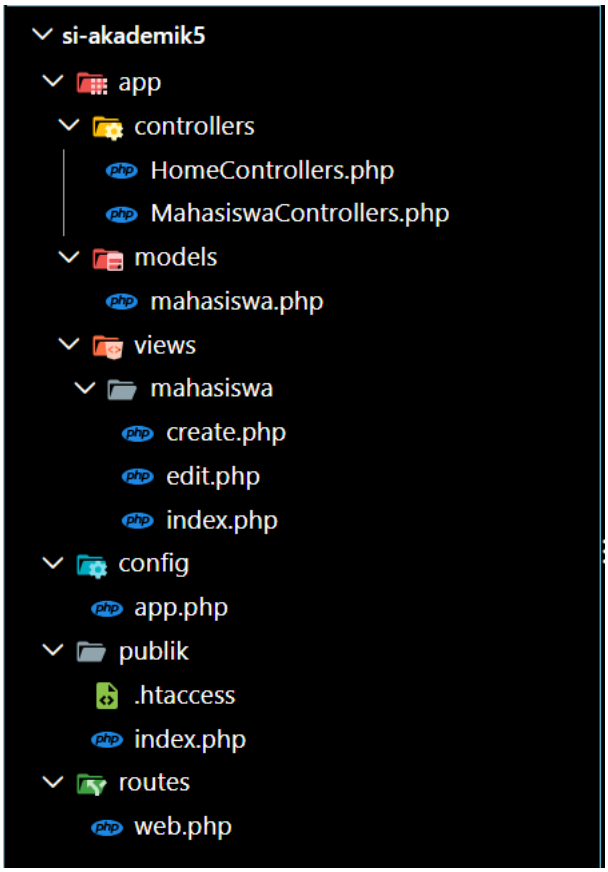
3.1 Hasil Praktik

A. Studi Kasus

Pada praktikum ini dilakukan implementasi routing sederhana pada proyek Si-Akademik yang berada pada:

`"C:\laragon\www\BKPM\si-akademik5"`

Struktur file yang digunakan adalah:



Gambar 1 Struktur File

File `.htaccess` digunakan untuk mengarahkan permintaan URL menuju `index.php`.

Kode `publik/.htaccess`:

```
RewriteEngine On

RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
```

```
RewriteRule ^ index.php [QSA,L]
```

Selanjutnya dibuat file `routes/web.php` untuk menentukan hubungan antara URL dengan controller dan method.

Kode `routes/web.php`:

```php
<?php

return [
    'GET' => [
        '/'                 => 'HomeController@index',
        '/login'            => 'AuthController@loginForm',
        '/logout'           => 'AuthController@logout',
        '/mahasiswa'        => 'MahasiswaController@index',
        '/mahasiswa/create' => 'MahasiswaController@create',
    ],
    'POST' => [
        '/login'            => 'AuthController@login',
    ],
];
```

Kemudian dibuat `HomeController.php` sebagai controller untuk halaman utama.

```php
<?php

class HomeController
{
    public function index()
    {
        echo "Halaman Home";
    }
}
```

Selain itu, dibuat `MahasiswaController.php` yang memiliki method `index()` dan `create()`.

```php
<?php

class MahasiswaController
{
    public function index()
    {
        echo "Halaman Data Mahasiswa";
    }
```

```
    public function create()
    {
        echo "Halaman Tambah Mahasiswa";
    }

    public function show($id)
    {
        echo "Detail Mahasiswa dengan ID: " .
htmlspecialchars((string) $id, ENT_QUOTES, 'UTF-8');
    }
}
```

File `publik/index.php` digunakan sebagai *front controller* yang menerima request dari pengguna dan menentukan controller serta method yang akan dijalankan.

Kode `publik/index.php`:

```
<?php

session_start();

require_once __DIR__ . '/../app/controllers/HomeControllers.php';
require_once __DIR__ . '/../app/controllers/AuthControllers.php';
require_once __DIR__ .
'/../app/controllers/MahasiswaControllers.php';

$routes = require __DIR__ . '/../routes/web.php';

$uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);

// Base path otomatis (tidak perlu diubah walau folder
dipindah/diganti nama)
// contoh hasil: /BKPM/si-akademik5/publik
$basePath = rtrim(str_replace('\\', '/',
dirname($_SERVER['SCRIPT_NAME'])), '/');
define('BASE_URL', $basePath);

if ($basePath !== '' && strpos($uri, $basePath) === 0) {
    $uri = substr($uri, strlen($basePath));
}

// Rapikan: "" atau "/login/" => "/" atau "/login"
$uri = '/' . trim($uri, '/');

$method = $_SERVER['REQUEST_METHOD'];
```

```php
// Route dengan parameter: GET /mahasiswa/{id}
$parts = explode('/', trim($uri, '/'));

if (
    $method === 'GET' &&
    count($parts) === 2 &&
    $parts[0] === 'mahasiswa' &&
    is_numeric($parts[1])
) {
    $controller = new MahasiswaController();
    $controller->show($parts[1]);
    exit;
}

if (isset($routes[$method][$uri])) {

    [$controllerName, $methodName] = explode('@',
$routes[$method][$uri]);

    $controller = new $controllerName();
    $controller->$methodName();

} else {

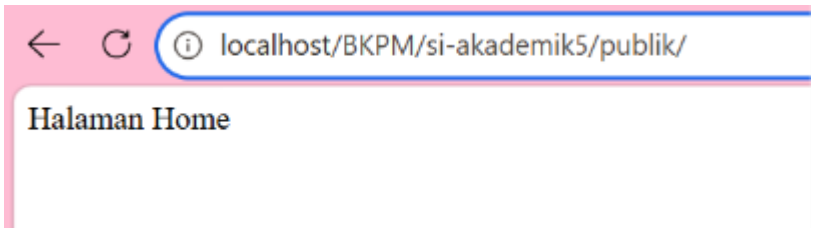
    http_response_code(404);
    echo "404 - Halaman Tidak Ditemukan";
}
```

Setelah seluruh file dibuat, Apache dijalankan melalui XAMPP. Kemudian aplikasi diuji melalui browser dengan beberapa URL.

URL:

`localhost/BKPM/si-akademik5/publik/`

menampilkan:

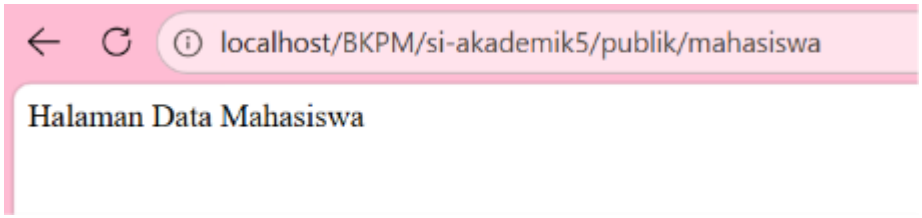


Gambar 2 Hasil pengujian halaman Home

URL:

`localhost/BKPM/si-akademik5/publik/mahasiswa`

menampilkan:

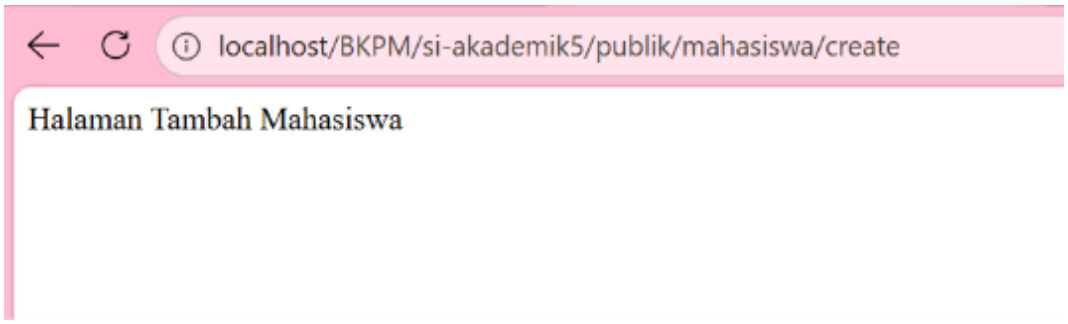


Gambar 3 Hasil pengujian URL `/mahasiswa`

URL:

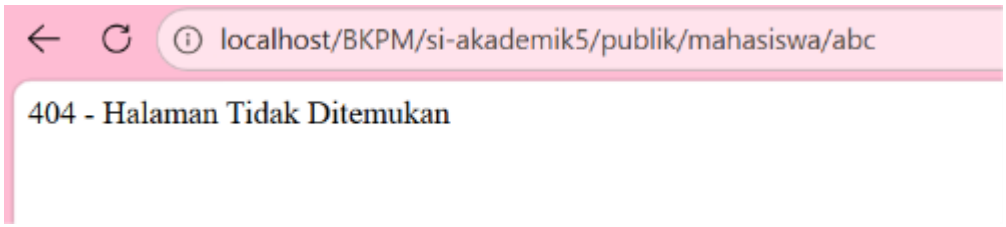
`localhost/BKPM/si-akademik5/publik/mahasiswa/create`

menampilkan:



Gambar 4 Hasil pengujian URL `/mahasiswa/create`

Selanjutnya dilakukan pengujian terhadap URL yang tidak tersedia, yaitu:
`localhost/BKPM/si-akademik5/publik/mahasiswa/abc`
Hasil pengujian menampilkan:



Gambar 5 Hasil pengujian URL yang tidak tersedia

B. Tugas Mandiri

Pada tugas mandiri ditambahkan dukungan parameter URL sederhana. Parameter digunakan untuk mengirimkan ID mahasiswa melalui URL.

Contoh URL yang digunakan:

localhost/BKPM/si-akademik5/publik/mahasiswa/5

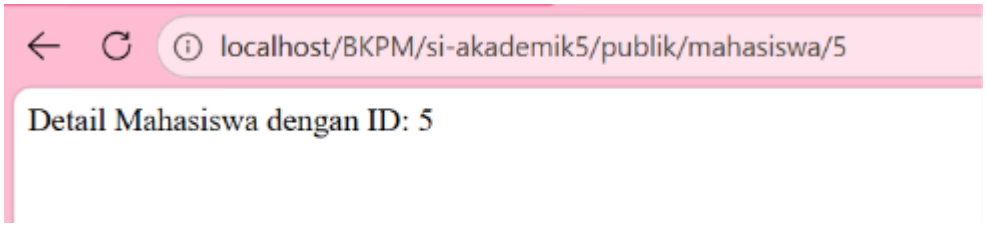
URL tersebut akan diproses menggunakan fungsi `explode()` sehingga bagian URL dapat dipisahkan menjadi nama resource dan ID.

Potongan kode yang digunakan:

```
// Route dengan parameter: GET /mahasiswa/{id}
$parts = explode('/', trim($uri, '/'));

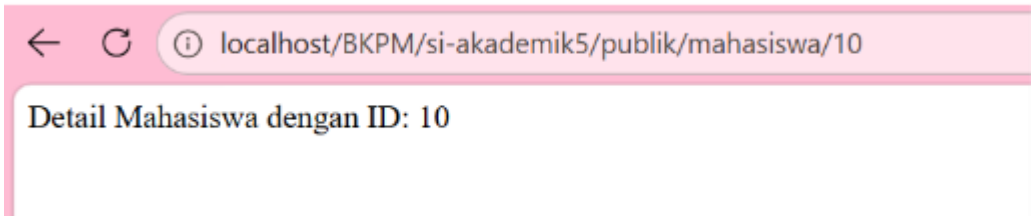
if (
    $method === 'GET' &&
    count($parts) === 2 &&
    $parts[0] === 'mahasiswa' &&
    is_numeric($parts[1])
) {
    $controller = new MahasiswaController();
    $controller->show($parts[1]);
    exit;
}
```

menampilkan:



Gambar 6 Hasil pengujian parameter URL `/mahasiswa/5`

Pengujian juga dapat dilakukan dengan ID yang berbeda, misalnya:
localhost/BKPM/si-akademik5/publik/mahasiswa/10
dan menghasilkan:



Gambar 7 Hasil pengujian parameter URL `/mahasiswa/10`

Hal tersebut menunjukkan bahwa parameter pada URL dapat berubah sesuai ID yang diberikan.

BAB 4. PEMBAHASAN

4.1 Pembahasan

Pada praktikum ini dilakukan implementasi routing sederhana pada aplikasi web menggunakan PHP dan Apache. Routing digunakan untuk menghubungkan URL yang diakses pengguna dengan controller dan method tertentu. Dengan adanya routing, pengguna tidak perlu mengakses file controller secara langsung.

Proses dimulai ketika pengguna memasukkan URL pada browser. Browser mengirimkan request kepada web server Apache. File `.htaccess` kemudian mengarahkan request tersebut menuju `index.php` sebagai *front controller*. Selanjutnya, `index.php` membaca URL yang diminta dan mencocokkannya dengan aturan yang terdapat pada `routes/web.php`.

Sebagai contoh, ketika pengguna mengakses URL `/mahasiswa`, routing akan mencocokkannya dengan aturan:

```
'/mahasiswa' => 'MahasiswaController@index'
```

Kemudian `MahasiswaController` dibuat dan method `index()` dijalankan sehingga menghasilkan tulisan "Halaman Mahasiswa".

Hal yang sama juga diterapkan pada URL `/mahasiswa/create`. URL tersebut diarahkan ke method `create()` pada `MahasiswaController`. Apabila URL yang dimasukkan tidak terdapat dalam daftar routing, aplikasi akan memberikan respons dengan kode HTTP 404 dan menampilkan pesan "404 - Halaman Tidak Ditemukan".

Pada tugas mandiri, routing dikembangkan dengan menambahkan parameter pada URL. URL `/mahasiswa/5` diproses menggunakan fungsi `explode()` untuk memisahkan bagian URL. Angka `5` kemudian digunakan sebagai parameter `$id` dan dikirimkan ke method `show($id)`. Dengan demikian, method `show()` dapat menerima ID yang berbeda sesuai dengan parameter yang diberikan pada URL.

Implementasi tersebut menunjukkan bahwa routing dapat membuat pengelolaan URL menjadi lebih terstruktur. Selain itu, penggunaan `.htaccess` dan *front controller* membuat request aplikasi diproses melalui satu pintu sebelum diteruskan ke controller yang sesuai.

BAB 5. PENUTUP

5.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa routing digunakan untuk mengatur hubungan antara URL dengan controller dan method pada aplikasi PHP. `.htaccess` digunakan untuk mengarahkan request menuju `index.php` sebagai front controller`, sehingga request dapat diproses melalui sistem routing.

Hasil pengujian menunjukkan bahwa URL `/`, `/mahasiswa`, dan `/mahasiswa/create` dapat diarahkan ke controller dan method yang sesuai, sedangkan URL yang tidak terdaftar menghasilkan respons 404. Pada tugas mandiri, parameter URL `/mahasiswa/5` juga berhasil digunakan untuk memanggil `show($id)` dengan nilai ID sebesar 5.

Dengan demikian, praktikum ini memberikan pemahaman mengenai dasar routing, `.htaccess`, *front controller*, serta penggunaan parameter sederhana pada URL dalam aplikasi web berbasis PHP.